Detector Lifetime Under Delayed Labels: A Pre-Registered Time-Aware Study on Android and Windows PE Malware

Yathindra K B
VISTAS, India
yathindrakb25@gmail.com

***Abstract*—A malware detector decays as malware changes, and an analyst team can slow the decay only by labelling samples that arrive late and are limited in number. We study malware detector lifetime under label delay, a fixed analyst budget and conformal drift monitoring in a single time-aware protocol, reported with A-AUT, on Android and Windows PE data. The protocol enforces the three TESSERACT constraints as assertions that raise, simulates a label queue in which no policy, selector or monitor can read a held-back label, and was pre-registered before the results of each of three grids (54, 56 and 76 cells) were produced. On two public feature datasets (LAMDA and BODMAS) and two detectors, we find small effects: retraining on bought labels helps a little on LAMDA, late labels cost a little, selecting labels by uncertainty or credibility is above random selection by about one to two hundredths of A-AUT, and no drift monitor beats retraining every third slot. Where a result depends on a choice, such as the random-split comparator, we report both readings. Code, claims ledger and registrations are public; every reported number maps to a file and a commit.**

***Index Terms*—malware detection, concept drift, label delay, active learning, conformal prediction, time-aware evaluation, reproducibility**

I. Introduction

A learned malware detector is trained on yesterday's samples and used on tomorrow's. Its F1 score falls as the samples change, and the standard remedy is to retrain on newly labelled data. Two facts make this harder than the textbook picture. First, labels are not free: an analyst team can label only a limited number of samples per day. Second, labels are not instant: a sample queried today may receive its label days or weeks later. How long a detector stays useful therefore depends on three things at once, the delay, the budget and the rule that decides when and what to label.

Prior work evaluates these ingredients separately: time-aware experiment design and the AUT metric [1], [2], [3], conformal drift detection [4], retraining under a labelling budget [5], [6], and the effect of label delay [7]. This paper puts them into one protocol and one codebase. We state the claim as narrowly as we can:

> *We study malware detector lifetime under label delay, a fixed analyst budget and conformal drift monitoring in a single time-aware protocol, reported with A-AUT, on Android and Windows PE data.*

This is a statement of what was measured, not of priority. The search behind it was limited, and neighbouring work covers subsets of the same ingredients.

Research questions. RQ1: how far does a random split overstate the time-aware score? RQ2: what do delay and budget cost or buy, and is the budget response concave? RQ3: does selecting labels by uncertainty or conformal credibility beat random selection, and does the advantage shrink under delay? RQ4: does a drift monitor that triggers retraining beat retraining on a fixed period at the same label cost?

Contributions. (1) A protocol with C1–C3, label delay and analyst budget as code, with a leakage canary and a mutation matrix in which all 15 deliberately broken constraints are caught [W-07]. (2) A simulator in which label delay lives in one module and held-back labels are sealed from selectors, policies and monitors, pinned by a spy test. (3) Three pre-registered grids, run to completion without a failed cell, with every decision made before or after the results marked as such. (4) A set of small or null effects, stated with intervals, and the audit trail behind each number. We make no claim that a better detector results.

II. Background

Time-aware evaluation. TESSERACT [1] identifies spatial and temporal bias and proposes three constraints: C1, never train on the future; C2, evaluate each slot using only samples from that slot; C3, keep a realistic malware ratio in the test set. AUT is the normalised area under the F1-versus-time curve. The extended version [2] adds case studies in Windows PE and PDF, and Chow et al. [3] show that factors beyond these constraints still cause discrepancies and recommend reporting AUT over rolling windows (A-AUT), which we do.

Drift and conformal monitors. Conformal evaluation turns a classifier's score into a p-value relative to a calibration set and uses it to detect drift in malware classification [4]. Its guarantees assume exchangeable samples [8], which a drifting stream violates, and the label-conditional p-values need labels that a delay queue releases late.

Retraining under a budget and under delay. Continuous and active learning reduce the cost of keeping Android detectors current [5], [9], [6]. Botacin and Gomes study label delays in detection pipelines and show that analysis-queue

TABLE I
THE TWO DATASETS AS USED (FEATURE FILES ONLY).

	LAMDA	BODMAS
Platform	Android	Windows PE
Period	2013 to 2025	Aug 2019 to Sep 2020
Samples	1,008,381	57,293 mal., 77,142 ben.
Features	Drebin-style, sparse	2,381, EMBER-style
Slot	one month	two weeks
Licence	MIT	code BSD-2-Clause[a]
Windows in E2	16 [E-28]	20 distinct slots [E-12]

[a] The dataset files carry no stated licence; they are used as published and not redistributed.

limits leave a large exposure [7]; their figures come from their setting and none is reused here.

Neighbours on drift, selection and delay. Gama et al. frame concept drift as a change in the relation between inputs and target [10], and Ceschin et al. argue that drift, evolution, delayed labels and adversarial pressure must be treated together in security [11]. For Windows PE, Li et al. study adaptation to real drifted malware from minimal samples [12]; for Android, DREAM and TIF address drift by explanation-based adaptation and by learning temporally invariant features [13], [14]. ActDroid treats Android detection as an active online problem [15], ADAPT lowers the labelling cost of frequent retraining by pseudo-labelling [16], and DRMD learns to reject samples for manual labelling [17]. Cerqueira et al. benchmark 14 drift detection methods on seven real-world datasets with timing-aware criteria [18], which is the way our monitors are scored (alarm precision, recall and delay against drop events defined from true labels). We report these works at the level of their abstracts; none of their figures is reused, and we do not claim that the combination studied here is absent from the literature.

III. PROTOCOL

A. Data

We use pre-extracted feature files only; no sample is downloaded, executed or redistributed. LAMDA [19] is an Android dataset of 1,008,381 apps in Drebin-style sparse features; its month comes from the AndroZoo `added` date, and its labels follow a VirusTotal rule (no detection is benign, four or more is malware, one to three are discarded) [D-15]. BODMAS [20] holds 57,293 malware and 77,142 benign Windows PE files in 2,381-dimensional EMBER-style features over about thirteen months. Time is cut into equal slots by first-seen date: a month for LAMDA, two weeks for BODMAS.

B. Slots, windows and constraints

A window trains on the first slots and then steps through the following evaluation slots one at a time; the window geometry is fixed by an ADR before any grid. C1 and C2 are checked by assertions that raise on violation, and C3 subsamples malware to a fixed 10% ratio per slot (with a cap-and-flag rule when a slot has too little malware), with a sensitivity check at another ratio. A slot that already has less malware than the target keeps all its rows and is flagged rather than dropped; 31 of the 62 LAMDA headline months are below 10% malware. [W-01]

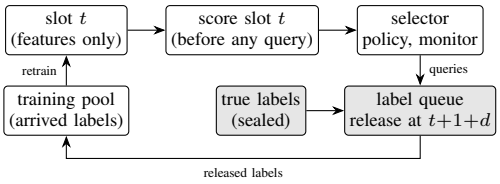


Fig. 1. One slot of the simulator. Selectors, policies and monitors see only the features of the slot; the queue is the only module that applies a delay and the only reader of true labels besides the scorer.

C. Metric

For per-slot F1 values $f_1, \ldots, f_N$, $\mathrm{AUT} = \frac{1}{N-1}\sum_{k=1}^{N-1}\frac{f_k+f_{k+1}}{2}$. A-AUT is the mean and standard deviation of AUT over rolling windows. We state intervals as two-stage bootstrap percentile intervals: moving blocks of windows, then seeds within windows, 10,000 replicates [P-08]. A contrast is *detected* when its 95% interval excludes zero in the hypothesised direction, and *equivalent* when the whole interval lies within ± 0.01 of A-AUT [P-10].

D. Delay, budget, policies, selectors, monitors

Delay and budget. A sample queried in slot t is usable for training from slot $t+1+d$, with $d \in \{0, 1, 2, 3\}$. The budget is 0.5, 1, 2 or 5% of the slot, and we always report the absolute count next to the percentage. **Policies.** *Never* trains once; *periodic* retrains every third slot; *monitor-triggered* retrains when a monitor alarms. Retraining uses only labels that have arrived. **Selectors.** *Random*; *uncertainty* (closest to 0.5); *credibility* (lowest Mondrian conformal p-value). A random slice is reserved for calibration. **Monitors.** Four label-free monitors behind one alarm interface: a Kolmogorov–Smirnov test on top features, MMD, a domain classifier, and a p-uniformity test on conformal scores. **Detectors.** Logistic regression and LightGBM at fixed hyperparameters; the point is the protocol, not a better detector.

IV. SIMULATOR AND ASSURANCE

Sealed labels. Label delay lives only in the queue module. A label queried in slot t is released exactly once, ignored and counted if its release slot falls after the evaluation window, and never readable before then; tests cover $d \in \{0, 1, 2, 3\}$ and 150 property-based scenarios [S-01]. Selectors, policies and monitors receive an opaque view of a slot; a spy test and a label-invariance test show they cannot reach a held-back label. A slot is scored before any label queried in it can exist.

Mutation checks. We broke the slicer and constraints in 15 documented ways (for example one training sample as new as the test slot, shuffled timestamps, an inclusive end of the slot window) and confirmed that the real test suites catch every one [W-07]. This is a check of code behaviour, not evidence about detectors. A leakage canary shows AUT moving toward the random-split score when leakage is planted in seeded drift.

TABLE II
HEADLINE CONTRASTS, A-AUT DIFFERENCE WITH 95% INTERVAL (LAMDA UNLESS STATED). LR IS LOGISTIC REGRESSION, LGBM IS LIGHTGBM.

Contrast	LR	LGBM
Periodic minus never, d=0, 5% [E-30]	0.017 [0.007, 0.024]	0.020 [0.012, 0.022]
Delay 0 minus delay 3, 5% [E-30]	0.011 [0.005, 0.017]	0.013 [0.008, 0.015]
Periodic minus never, BODMAS [E-29]	0.003	none
Selection over random, d=0 [E-43]	0.0109 to 0.0177	
Selection over random, d=3 [E-43]	0.0015 to 0.0095	
Selection × delay interaction [E-42]	0.0061 to 0.0118 (8 of 8 positive)	

TABLE III
E2 ON LAMDA: A-AUT(F1), MEAN OVER 16 WINDOWS (SD OVER WINDOWS), FIVE SEEDS, PERIODIC RETRAINING, MEAN LABELS PER SLOT [E-28].

Arm	Labels/slot	LR	LGBM
never	0	0.679 (0.058)	0.760 (0.069)
d=0, 0.5%	21.8	0.682 (0.058)	0.763 (0.068)
d=0, 1%	43.6	0.683 (0.058)	0.767 (0.066)
d=0, 2%	87.3	0.687 (0.055)	0.772 (0.064)
d=0, 5%	218.3	0.694 (0.050)	0.781 (0.059)
d=3, 5%	218.3	0.684 (0.054)	0.767 (0.065)

TABLE IV
E3 ON LAMDA AT d=0, 2% BUDGET, SEEDS 0 TO 4: SHARE OF MALWARE AMONG THE LABELS THAT ARRIVED, AND A-AUT, BY SELECTOR [E-47].

	LR		LGBM	
Selector	malware	A-AUT	malware	A-AUT
random	0.085	0.6676	0.085	0.7647
credibility	0.193	0.6829	0.303	0.7806
uncertainty	0.266	0.6870	0.401	0.7812

Reproducibility. Every run has one YAML configuration; its hash covers the config, the code version and the lock file. Per-slot predictions are stored. A smoke experiment on a synthetic fixture runs from an installed wheel, and `driftwatch reproduce table1` reproduces the first table on BODMAS: on a clean tree the time-aware columns matched to four decimals [E-24,E-82].

V. EXPERIMENTS AND PRE-REGISTRATION

Experiments E0 to E4 were run, and three of them were pre-registered. E0 and E1 (baselines and the label-influence diagnostic) were run and read *before* the first registration, so RQ1 is a registered reading of numbers already known, and its primary comparator was chosen after both numbers were seen [E-18]. E2 (delay × budget, 54 cells) was registered under tag `prereg-v1`, E3 (selection under delay, 56 cells) under `prereg-v2`, and E4 (drift monitors, 76 cells) under `prereg-v3`, each before its first result. All 186 cells completed and none failed [E-26,E-40,E-48]. Seeds extended after reading (LAMDA seeds 5 to 9 of E3) and one setting stated after the results (the Holm floor of E4) are marked as such in the registrations and in the claims ledger [P-18]. E5 (cross-platform repeat) and E6 (EMBER2024) were not run.

VI. RESULTS

We report A-AUT differences with 95% intervals; standard deviations over LAMDA windows are 0.050 to 0.069 [E-28]. Figure 2 shows the per-slot curves and Table II gathers the headline contrasts.

RQ2: delay and budget. Periodic retraining with 5% of each month's samples labelled (about 218 labels per slot) is above never retraining by 0.017 [0.007, 0.024] for logistic regression and 0.020 [0.012, 0.022] for LightGBM; on BODMAS the difference is 0.003 for logistic regression and none for LightGBM [E-30,E-29]. A delay of three slots instead of zero costs 0.011 and 0.013 at 5%; at 1% the cost is not detected for logistic regression and lies inside the 0.01 margin for LightGBM. Moving from 0.5% to 5% adds 0.013 and 0.017 on LAMDA. Diminishing returns are detected in one stratum of four: per 100 extra labels per slot the gain falls from 0.017 to 0.007 for LAMDA LightGBM [E-34]. Whether the delay cost grows with budget was not a registered contrast and is not claimed. On BODMAS there is one useful refit per window, so at $d = 3$ periodic retraining equals never retraining by construction and its delay numbers measure a delay in a single refit [P-12].

RQ1: the random-split gap. Under the registered per-slot comparator the gap is 0.03 to 0.04 on LAMDA and 0.006 to 0.035 on BODMAS, below the registered 0.05 band, while the LAMDA LR interval still contains 0.05; under the window-level comparator it is 0.14 to 0.15 on LAMDA [E-35]. The two readings differ because of how the comparator matches class counts, and the primary one is biased downwards on LAMDA [E-18]. We therefore do not claim that the gap is caused by temporal leakage alone.

RQ3: selection. On LAMDA, with ten seeds and 43.6 and 87.3 labels per slot, uncertainty or credibility selection is above random selection at equal labels by 0.0109 to 0.0177 with no delay and 0.0015 to 0.0095 after a delay of three slots; at $d = 3$ three of eight advantages, all logistic regression, have intervals that include zero [E-43]. The interaction is positive in all eight rows, 0.0061 to 0.0118, under a Wilcoxon test on seed means that is conditional on the realised windows [E-42]. The advantage is not separated from a change in class mix of the labels that arrive [E-47], and we do not show that credibility adds anything over uncertainty. The calibration-contaminated trap arm never retrains, so it is not read as evidence about contamination [E-45].

RQ4: monitors. None of the 32 monitor-minus-periodic contrasts has an interval above zero; logistic regression with the KS monitor is below periodic retraining by 0.0089 [−0.0225, −0.0002] at $d = 0$ and 0.0052 [−0.0136, −0.0007]

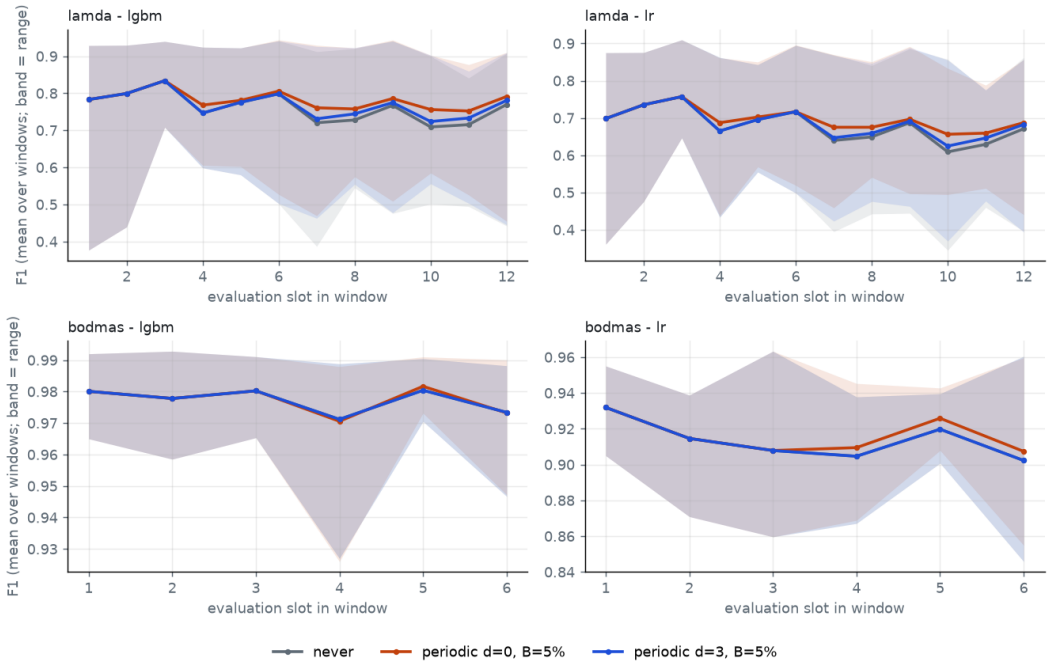


Fig. 2. E2: per-slot F1 inside a window (mean over windows, band is the range over windows) for never retraining, periodic retraining with no delay and periodic retraining with a delay of three slots, both at 5% of the slot [E-26]. On LAMDA the never-retrained curve falls below the retrained ones from about the fourth evaluation slot; on BODMAS they are almost the same.

TABLE V
E4 MONITOR QUALITY ON LAMDA WITH LIGHTGBM, MONITORS OBSERVING THE NEVER-RETRAINED ARM: ALARM RATE, ALARM PRECISION (95% INTERVAL) AGAINST THE PRECISION OF ALARMING IN EVERY SLOT (0.443), AND ALARM RECALL, FOR DROPS OF AT LEAST 0.05 BELOW THE TRAILING THREE-SLOT MEAN [E-63].

Monitor	Alarm rate	Precision	Recall
KS (top features)	0.617	0.462 [0.354, 0.546]	0.743
MMD	0.796	0.449 [0.328, 0.546]	0.942
Domain classifier	0.997	0.444 [0.327, 0.529]	1.000
p-uniformity	0.562	0.461 [0.315, 0.555]	0.796

at $d = 3$ on LAMDA, a descriptive reading without family-wise control [E-59,E-60,P-18]. At the same number of refits the timing of the alarms made no difference larger than 0.01 on LAMDA [E-61], but the monitors alarmed in most slots, so where an alarm fires every slot the matched control does not test timing [E-63]. This is a negative result at one budget and one frozen alarm level.

VII. READING THE RESULTS TOGETHER

Three observations hold across the four questions. First, the size of every effect is small next to the spread over windows, and the larger effects sit on LAMDA, the dataset with a longer series and more refit events per window; BODMAS offers one useful refit per window, so most of its contrasts are null or collapse into one comparison. Second, the label-selection result and the budget result are of the same order (about one to two hundredths of A-AUT), which suggests that at these budgets the choice of which labels to buy matters about as much as buying more, but the selector advantage is confounded with the class prior of the arrived labels: selection raises the malware share of the pool from 0.085 to between 0.19 and 0.40 on LAMDA (Table IV), and a class-matched random baseline would separate the two effects. Third, the monitors alarm in most slots (alarm rates of 0.56 to 0.997 in Table V) and their precision stays near that of alarming every slot, so at the alarm level frozen before the run they are not informative about when to retrain; a different level, a larger sample per slot or a label-using monitor are untested.

What was fixed before the results. The window geometry and the size gate (ADR-0005), the metric thresholds (ADR-0006), the pool composition and the budget rule (ADR-0007), the E2 design and the RQ1 comparator (ADR-0008), the rule that calibration rows are never training rows (ADR-0009) and the monitor inputs and reference (ADR-0010) were decided and recorded in ADRs before the results of the grids they govern (E2 to E4), with the exception of the RQ1 comparator, which was chosen after E0 and E1 had been read. We emphasise this because the final conclusions are null or small: a null result is credible only if the analysis could not have been chosen to produce it.

VIII. THREATS TO VALIDITY AND LIMITS

Small effects. Effects are hundredths of A-AUT against window standard deviations of 0.05 to 0.07 [E-28]; we do not claim that any of them matters operationally. **Few independent blocks.** LAMDA's 16 windows are about four independent blocks, seeds are draws of a subsample and not

replicates of a model, so intervals are wide and their coverage was not measured [P-12]. The registered scheme resamples windows and then seeds within them; the statistical literature on hierarchical data favours resampling the top level only [21], [22], and our statement that the two-stage interval is wider rests on a synthetic check rather than on those sources. **Labels.** LAMDA's labels are antivirus-centric and depend on a VirusTotal scan date the dataset does not publish, so a label may reflect later knowledge than the collection date, a leakage we cannot remove [D-16]. BODMAS's paper says families come mostly from multi-vendor antivirus verdicts analysed with in-house scripts and does not say how the binary label was decided [20]. **Class prior.** Per-slot F1 moves with the slot's malware share, correlating with the achieved ratio at 0.47 to 0.55 on LAMDA [E-66], so part of the apparent decay is class prior; a prior-matched baseline and a threshold-free score are computable from stored predictions and were not registered. **Simulated delay and analyst.** Delay is a whole number of slots, the same for every label, and a queried label is the dataset's truth returned without error. Neither dataset records when a label became known. **One machine, two datasets, two detectors.** All runs are on one laptop, and a difference between datasets is not a finding about platforms. **Review status.** Code and text were produced with an AI assistant and reviewed by other instances of the same model family; no person has yet read every chapter line by line.

IX. Ethics and availability

The work describes how detectors decay; it contains no malware, no decompiled code, no evasion tooling and no adversarial-example generation. Knowing when retraining pays could in principle help an adversary time an attack, but the study measures natural drift on public data, offers no method to cause drift and finds small effects.

The code (MIT), the configurations, the claims ledger, the registrations, the decision records and the aggregate tables are public at https://github.com/Yathindra25KB/driftwatch-lifetime. The package is `driftwatch-lifetime` on PyPI, and each release is archived on Zenodo (concept DOI 10.5281/zenodo.24050864). LAMDA, BODMAS and their licences are not redistributed; the repository holds only a synthetic fixture. Per-slot predictions are hashed in the repository and not yet deposited.

Use of AI assistance. The code and the drafts of this paper were produced with Claude Code (Anthropic) under the author's direction; the author decided every scope, registration and release step. The wording that a venue requires for disclosing such assistance must be checked before submission.

X. Conclusion

Under one time-aware protocol with simulated delay, a fixed budget and four drift monitors, detector lifetime on LAMDA and BODMAS responds to retraining and to label timing by hundredths of A-AUT, selection by uncertainty or credibility adds a little over random selection and less after a delay, and no drift monitor beat retraining every third slot. The BODMAS series is too short to separate delay from refit counts. The contributions that outlast these numbers are the protocol, its assertions and audits, and a record in which every choice made before or after seeing results is labelled. Future work needs a new pre-registration each: other monitor settings, a longer BODMAS window, label noise and measured delay, a trap arm that also trains on released labels, class-prior controls, a registered lifetime measure, a label-using monitor, a cross-platform repeat and EMBER2024.

References

[1] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time," in *USENIX Security Symposium*, 2019, arXiv v4 corresponds to the USENIX Security 2019 version and corrects a typo in Eq. (4) (an extra 1/N factor).

[2] Z. Kan, S. McFadden, D. Arp, F. Pendlebury, R. Jordaney, J. Kinder, F. Pierazzi, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time (extended version)," 2024, v1 2 Feb 2024, v2 9 Apr 2025.

[3] T. Chow, M. D'Onghia, L. Linhardt, Z. Kan, D. Arp, L. Cavallaro, and F. Pierazzi, "Beyond the TESSERACT: Trustworthy dataset curation for sound evaluations of Android malware classifiers," 2025.

[4] F. Barbero, F. Pendlebury, F. Pierazzi, and L. Cavallaro, "Transcending transcend: Revisiting malware classification in the presence of concept drift," in *IEEE Symposium on Security and Privacy*, 2022.

[5] Y. Chen, Z. Ding, and D. Wagner, "Continuous learning for Android malware detection," in *USENIX Security Symposium*, 2023.

[6] S. K. Amalapuram, B. Shresta, S. R. m. Chebiyam, B. R. Tamma, and S. S. Channappayya, "SEED: Semi-supervised continual malware detection for tackling concept drift on a budget," 2026.

[7] M. Botacin and H. Gomes, "Towards more realistic evaluations: The impact of label delays in malware detection pipelines," *Computers & Security*, vol. 148, p. 104122, 2025.

[8] G. Shafer and V. Vovk, "A tutorial on conformal prediction," 2008, journal of Machine Learning Research 9:371–421, 2008.

[9] L. Minnei, C. Manca, G. Piras, A. Sotgiu, M. Pintor, D. Ghiani, D. Maiorca, G. Giacinto, and B. Biggio, "Label-efficient training updates for malware detection over time," 2026.

[10] J. Gama, I. liobait, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, 2014.

[11] F. Ceschin, M. Botacin, A. Bifet, B. Pfahringer, L. S. Oliveira, H. M. Gomes, and A. Grégio, "Machine learning (In) security: A stream of problems," 2023, digital Threats 2023, DOI 10.1145/3617897 (arXiv record).

[12] A. S. Li, A. Iyengar, A. Kundu, and E. Bertino, "Revisiting concept drift in windows malware detection: Adaptation to real drifted malware with minimal samples," 2024, nDSS 2025 (listed on the NDSS 2025 accepted-papers page; the arXiv record carries no venue).

[13] Y. He, J. Lei, Z. Qin, K. Ren, and C. Chen, "Combating concept drift with explanatory detection and adaptation for Android malware classification," 2024, accepted to appear at ACM CCS 2025 (arXiv comment).

[14] X. Zheng, S. Yang, E. C. H. Ngai, S. Jana, and L. Cavallaro, "TIF: Learning temporal invariance in Android malware detectors," 2025, accepted by IEEE Transactions on Software Engineering (arXiv comment).

[15] A. Muzaffar, H. Ragab Hassen, H. Zantout, and M. A. Lones, "ActDroid: An active learning framework for Android malware detection," 2024.

[16] M. T. Alam, A. Piplai, and N. Rastogi, "ADAPT: A pseudo-labeling approach to combat concept drift in malware detection," 2025.

[17] S. McFadden, M. Foley, M. D'Onghia, C. Hicks, V. Mavroudis, N. Paoletti, and F. Pierazzi, "DRMD: Deep reinforcement learning for malware detection under concept drift," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 2, 2026, pp. 854–862.

[18] V. Cerqueira, H. M. Gomes, M. Heyden, B. Pfahringer, and A. Bifet, "A framework for evaluating and benchmarking concept drift detection methods," 2026, accepted at KDD 2026 (arXiv comment).

[19] M. A. Haque, I. Hossain, M. M. Kamol, M. J. Alam, S. K. Amalapuram, S. Talukder, and M. S. Rahman, "LAMDA: A longitudinal Android malware benchmark for concept drift analysis," 2025, presented at ICLR 2026 (poster page iclr.cc/virtual/2026/poster/10011850).

[20] L. Yang, A. Ciptadi, I. Laziuk, A. Ahmadzadeh, and G. Wang, "BODMAS: An open dataset for learning based temporal analysis of PE malware," in *2021 IEEE Security and Privacy Workshops (SPW)*, 2021, pp. 78–84.

[21] A. C. Davison and D. V. Hinkley, *Bootstrap Methods and their Application*. Cambridge University Press, 1997, section 3.8 treats hierarchical data.

[22] S. Ren, H. Lai, W. Tong, M. Aminzadeh, X. Hou, and S. Lai, "Nonparametric bootstrapping for hierarchical data," *Journal of Applied Statistics*, vol. 37, no. 9, pp. 1487–1498, 2010.

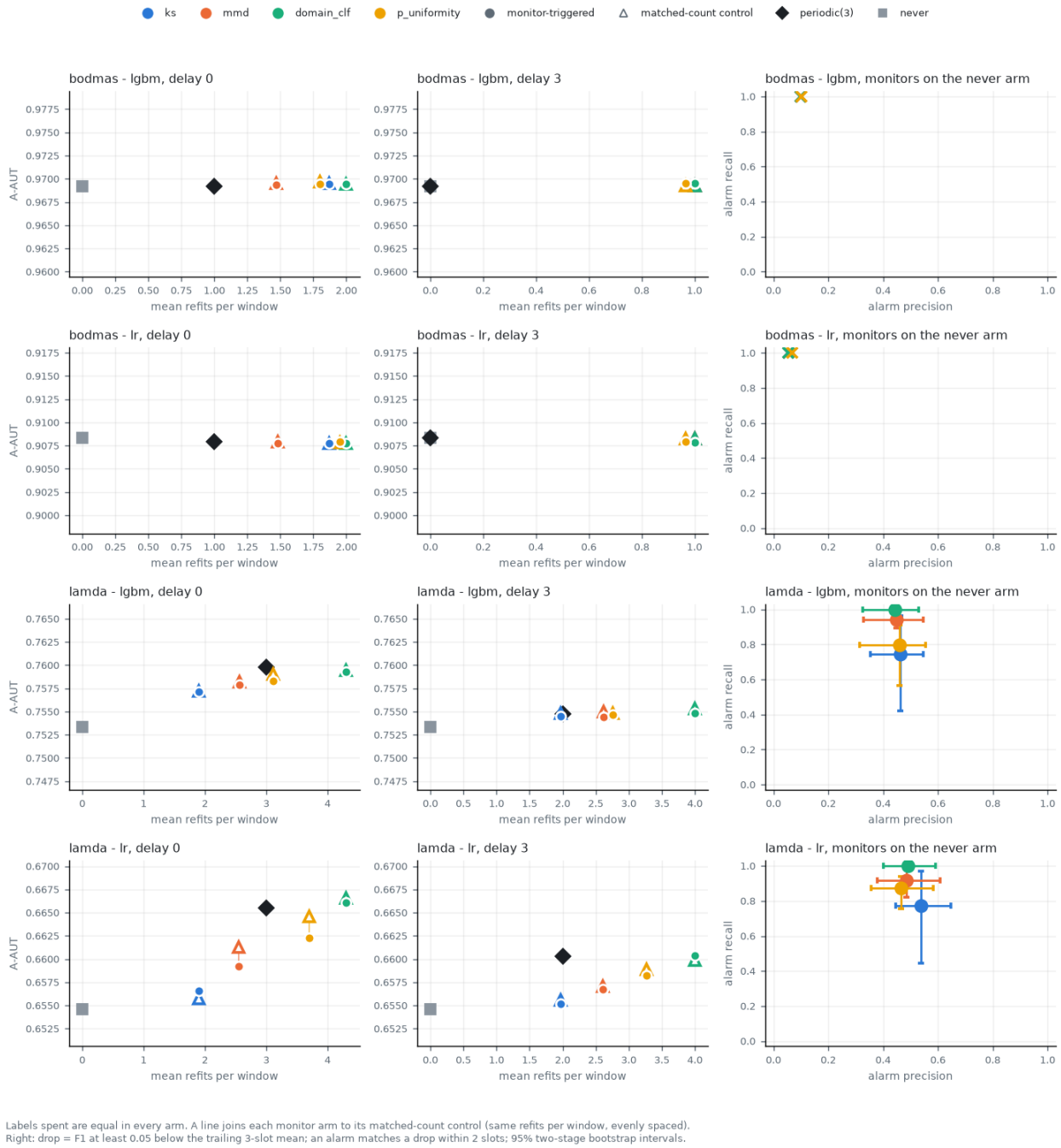


Fig. 3. E4: A-AUT against mean refits per window for each monitor-triggered arm, its matched-count control, periodic retraining every third slot and never retraining, at delay 0 and 3 (left and middle columns), and alarm precision against recall for each monitor (right column). Labels spent are equal in every arm; intervals are two-stage bootstrap 95% intervals.